Network Analysis Emergency Hospitals

#WebGIS #QGIS #UrbanPlanning #NetworkAnalysis #JavaScript #CSS #HTML #Leaflet

Data

- [ ] Road data (Lastkajen)source
- [ ] Population data (SCB) source
- [ ] Emergency hospital points (QuickOSM)
- [ ] County borders (SCB)source

Tools

1. QGIS

```
1. QNEAT3
2. SQL queries
```

2. VisualStudio Code

```
1. Javascript
2. CSS
3. HTML
```

3. Leaflet

Data prepping & cleaning

1. Hospital layer:

```
1. Load in with QuickOSM using the "quick query" function. set the Key as
'Emergency', and the Value as 'yes'
2. See which, if any of the three layers is most correct, otherwise create a join.
In this case, create centroids using the polygon layer, and then remove the excess
points.
3. Export as new layer.
```

2. County border layer

```
1. Import the layer.
2. Select the correct border.
3. Export the feature.
4. Add a buffer (10km).
```

2. Road layer

```
1. Add the layer to the map.
2. Use Select by location to select all roads in/touching the buffered version of
the
county border.
3. Export as a new layer.
4. Select all roads with a max speed value that is not NULL.
5. export as a new layer.
```

3. Population data

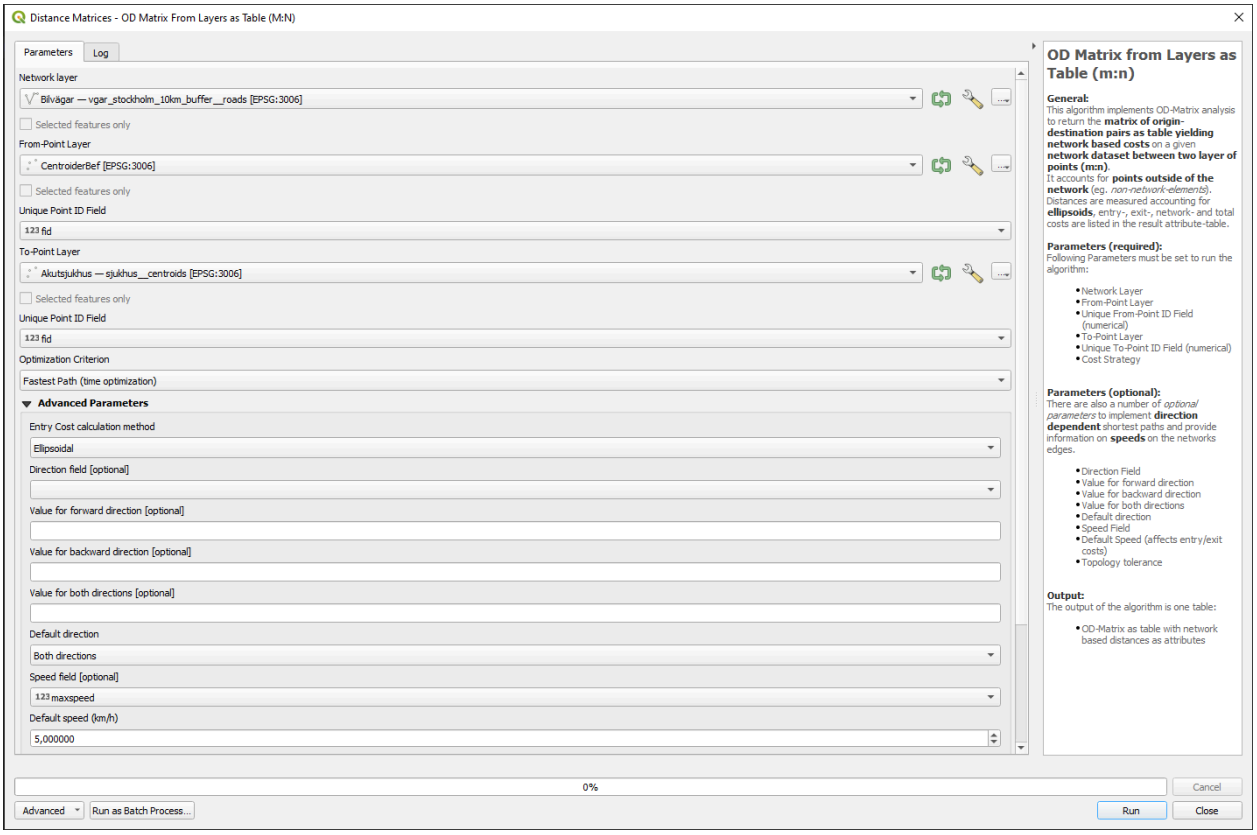
```
1. Import the layer to the map.
2. Use Select by location to select all cells in/touching the buffered version of
the county border.
3. export as a new layer.
4. Create centroids for each cell multipolygon (cell).
```

Network analysis

1. QNEAT3

- [ ] Use the OD Matrix from Layers as table m:n
    1. Set the Network Layer as the buffered road layer
    2. Set the From-Point Layer as the population centroids
    3. Set the To-Point Layer as the hospital centroids
    4. Set the Optimization Criterion as "Fastest Path (time optimization)"

5. Under Advanced Parameters, set the Speed field to "maxspeed" (or alternative name)



This returns a table with the route for each of the population centroids to each of the emergency hospitals.

2. Joining data

- [ ] Join the table with the emergency hospital layer setting the join field to "destination_id", and the Target field as fid. This lets the user see which hospital each travel path arrives at.

3. Data cleaning

To isolate only the fastest path from each population centroid, use a SQL query such as the following to create a new table:

```
"sum" = aggregate(

    @layer,

    'min',

    "sum",

    filter := "origin_id" = attribute(@feature, 'origin_id')

)
```

4. Joining data

1. Join the new table to the population centroid layer, setting 'origin_id as the Join field and 'fid' as the Target field. Now each centroid has the fastest travel path to the closest hospital.
2. Now join the population centroid layer with the population multipolygon layer setting both the Join field and Target field as 'Rutid_SCB'.
3. Create a new, duplicated layer from the population multipolygon layer for further processing.

5. Field creation

1. Rename the 'total_cost' field, which contains the travel time in seconds, to Time(sec).
2. Create a new field called Time(min) using the expression: `"Time(sec)/60"` in the field calculator.
3. Create a new field called TimeZones using the follwing expression:

```
   CASE

WHEN "time(sec)" <= 600 THEN '0-10 min'

WHEN "time(sec)" <= 900 THEN '10-15 min'

WHEN "time(sec)" <= 1200 THEN '15-20 min'

WHEN "time(sec)" <= 1800 THEN '20-30 min'

ELSE '>30 min'

END
```

6. Symbology

1. Change the symbology to "Categorized", setting the field 'TimeZones' as the value.
2. Choose appropriate colours for the five categories, such as deep green, lighter green, yellow, orange, and red.

7. Export

Either create a layout and export the data as a static map, or export the layer as in geoJSON format to allow for webGIS implementation.

WEBGIS

JavaScript

```
let geojsonLayer;
const map = L.map('map').setView([59.33, 18.07], 10);


L.tileLayer(
  'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
  {
    maxZoom: 19,
    attribution: '&copy; OpenStreetMap contributors'
  }
).addTo(map);


function getColor(zone){

    switch(zone){
        case "0-10 min":
            return "#228B22";

        case "10-15 min":
            return "#2ECC71";

        case "15-20 min":
            return "#F1C40F";

        case "20-30 min":
            return "#E67E22";

        case ">30 min":
            return "#E74C3C";

        default:
            return "#999999";
    }
   }


function highlightFeature(e){

    const layer = e.target;
    layer.setStyle({

        weight: 3,
        color: "#000",
```

```
        fillOpacity: 1
    });
    info.update(layer.feature.properties);


function resetHighlight(e){
    geojsonLayer.resetStyle(e.target);
    info.update();
    }


const info = L.control();


info.onAdd = function(){

    this._div = L.DomUtil.create('div','info');

    this.update();

    return this._div;
    };


info.update = function(props){

    this._div.innerHTML =
        props
        ? `<h4>${props.name}</h4>
           Travel time: ${Number(props.TimeMin).toFixed(1)} min<br>
           Population: ${props.beftotalt}`
        : 'Hover over a box';
        };


info.addTo(map);


let allData;


function updateZoneFilter(){
    const selectedZone =
    document.getElementById("zoneFilter").value;
    geojsonLayer.clearLayers();
    geojsonLayer.addData({
```

```
        type: "FeatureCollection",
        features:
        allData.features.filter(feature => {
            if(selectedZone === "all"){
                return true;
            }
            return feature.properties.Timezone2 === selectedZone;
        })
    })}


fetch('Data/ExportPoly2.geojson')
  .then(response => response.json())
  .then(data => {
    allData = data;

    geojsonLayer = L.geoJSON(data,{

    style: function(feature){
        return{
            fillColor: getColor(feature.properties.Timezone2),
            color: "#444",
            weight: 0.5,
            fillOpacity: 0.8
        };
    },

      onEachFeature: function(feature, layer) {
layer.on({
    mouseover: highlightFeature,
    mouseout: resetHighlight});
        layer.bindPopup(`
        <h3>${feature.properties.name}</h3>
        <b>Travel time:</b> ${Number(feature.properties.TimeMin).toFixed(1)}
min<br>
        <b>Population:</b> ${feature.properties.beftotalt} people`);
        }
        });

    geojsonLayer.addTo(map);
    createLayerControl();
    if(hospitalLayer){
        hospitalLayer.bringToFront();
    }

    map.fitBounds(geojsonLayer.getBounds());
```

```
  })

  .catch(error => console.error(error));


  const legend = L.control({
    position: 'bottomright'
    });


 legend.onAdd = function(map){

  const div = L.DomUtil.create('div','info legend');
    div.innerHTML =
        '<i style="background:#228B22"></i> 0-10 min<br>' +
        '<i style="background:#2ECC71"></i> 10-15 min<br>' +
        '<i style="background:#F1C40F"></i> 15-20 min<br>' +
        '<i style="background:#E67E22"></i> 20-30 min<br>' +
        '<i style="background:#E74C3C"></i> >30 min';
    return div;
    };


legend.addTo(map);


const hospitalIcon = L.icon({
    iconUrl: 'Images/HosIcon.png',
    iconSize: [32,32],
    iconAnchor: [16,32],
    popupAnchor: [0,-32]
    });


let hospitalLayer;


fetch('Data/AkutSjukhus.geojson')
.then(response => response.json())
.then(data => {
    hospitalLayer = L.geoJSON(data, {
        pointToLayer: function(feature, latlng) {
            return L.marker(latlng, {
                icon: hospitalIcon
            });
```

```
        },
        onEachFeature: function(feature, layer) {
            layer.bindPopup(`<b>${feature.properties.name}</b>`);
        }
    });
    hospitalLayer.addTo(map);
    createLayerControl();
    })


.catch(error => console.error(error));


let layerControl;


function createLayerControl() {
    if (!geojsonLayer|| !hospitalLayer) return;

   const overlays = {
        "Restidszoner": geojsonLayer,
        "Akutsjukhus": hospitalLayer
    };
    L.control.layers(null, overlays, {position: 'bottomleft'}).addTo(map);
    }


document
.getElementById("zoneFilter")
.addEventListener("change", updateZoneFilter);
```

`

HTML

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title> Accessability to Emergency Hospitals</title>

        <link rel="stylesheet" href="https://unpkg.com/leaflet/dist/leaflet.css"/>

        <link rel ="stylesheet"

        href="css/style.css">
```

```
    </head>

    <body>
        <div id="header">
          <h1>Map of Emergency Hospital Accessability</h1>
        </div>

        <div id = "controls">
            <select id = "ZoneFilter">
                <option value="all"> All zones</option>
                <option value="0-10 min"> 0-10 min</option>
                <option value="10-15 min"> 10-15 min</option>
                <option value="15-20 min"> 15-20min</option>
                <option value="20-30 min"> 20-30 min</option>
                <option value=">30 min"> >30 min</option>
            </select>
        </div>

        <div id="map"> </div>
        <script src="https://unpkg.com/leaflet/dist/leaflet.js"></script>
        <script src="JS/map.js"></script>
    </body>
</html>
```

CSS

```
html,

body,

#map {
    height: 100vh;
    width: 100%;
    margin: 0;
    padding: 0;
}


#header {

    position: absolute;
    top: 20px;
    left: 50%;
    transform: translate(-50%);
    background: white;
```

```
    padding: 10px 20px;
    border-radius: 25px;
    box-shadow: 0 2px 5px rgba(0,0,0,0.3);
    z-index: 1000;
    opacity: 0.9;

}


.info {
    background: white;
    padding: 8px 10px;
    padding:5px;
    border-radius:5px;
}


.legend{
    background:white;
    padding:10px;
    border-radius:5px;
    box-shadow: 0 0 15px rgba(0,0,0,0.2);
}


.legend i{

    width:18px;
    height:18px;

    float:left;
    margin-right:8px;

    opacity:0.8;
}


#controls{
    position: absolute;
    top: 10px;
    left: 50px;
    z-index: 1000;
    background: white;
    border-radius: 5px;
}
```